

**EXERCISE 11.10**

Prove that these two statements of the identity laws are equivalent.

**EXERCISE 11.11**

Prove that the identity laws hold for a monad of your choice.

**EXERCISE 11.12**

There's a third minimal set of monadic combinators: `map`, `unit`, and `join`. Implement `join` in terms of `flatMap`.

```
def join[A] (mma: F[F[A]]) : F[A]
```

**EXERCISE 11.13**

Implement either `flatMap` or `compose` in terms of `join` and `map`.

**EXERCISE 11.14**

Restate the monad laws to mention only `join`, `map`, and `unit`.

**EXERCISE 11.15**

Write down an explanation, in your own words, of what the associative law means for `Par` and `Parser`.

**EXERCISE 11.16**

Explain in your own words what the identity laws are stating in concrete terms for `Gen` and `List`.

11.5 *Just what is a monad?*

Let's now take a wider perspective. There's something unusual about the `Monad` interface. The data types for which we've given monad instances don't seem to have much